Distributed Transactions Overview

all or nothing, across services

Phase 4 videos on YouTube

In one line: when one business action changes data in several services or databases there's no single transaction to make it all-or-nothing; you choose between two-phase commit (atomic but blocking and coupled) and sagas (local transactions plus compensations, eventually consistent), or you redesign the boundaries.

1 The problem

A customer places an order. You must create the order, charge the card and reserve the stock. In a monolith with one database, that's one transaction: all or nothing. Split into order, payment and inventory services, each with its own database, and there's no transaction that covers all three. If the charge succeeds and the stock reservation fails, the customer pays for nothing.

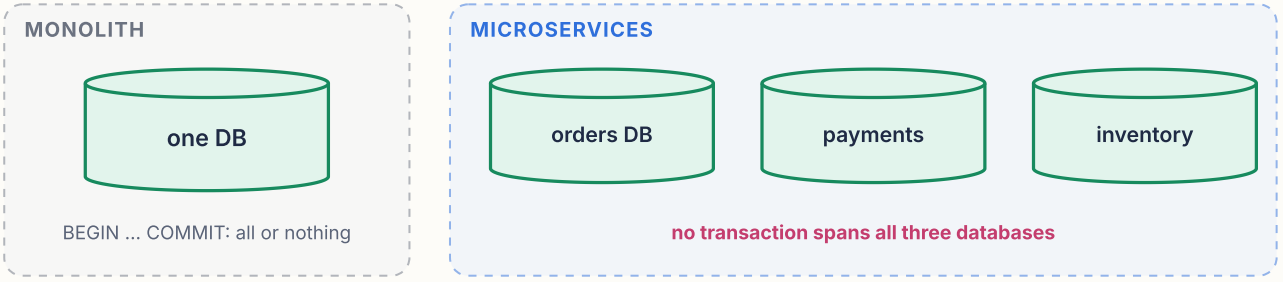


Figure 1. One database gives you ACID for free. Separate databases don't share a transaction.

2 What 'atomic' means across services

We want **atomicity** (all steps happen or none do, from the business point of view), and usually also **isolation** (nobody sees half-done states). Across services you can get atomicity with coordination, or settle for steps that are **eventually** all done or all undone. Full isolation across services is rarely affordable.



Figure 2. The four ways to approach it. Sagas with an outbox are the usual choice for microservices.

3 Two-phase commit

A **coordinator** asks every participant to **prepare** (lock and promise to commit), then tells all to **commit** only if all said yes (lesson 16). It's truly atomic, but locks are held across network round trips, a coordinator failure can leave participants blocked, and every participant must support XA. Good inside

one database cluster or a few cooperating databases; a poor fit for independent microservices and external APIs like a card processor.

4 Sagas

A **saga** splits the action into local transactions, one per service. Each step commits on its own. If a later step fails, earlier steps are undone by **compensating actions** (refund the card, release the stock). The system is briefly inconsistent, but it ends either fully done or fully compensated (lesson 17). To make each step reliable, services save state and publish events atomically with the **transactional outbox** (lesson 37), and consumers deduplicate (lesson 38).

WORKED EXAMPLE: ORDER, PAYMENT, INVENTORY AS A SAGA

The order service creates the order as **PENDING** and writes an `OrderCreated` event to its outbox in the same transaction. The payment service consumes it, charges the card with an idempotency key, and emits `PaymentCompleted`. The inventory service tries to reserve stock and fails: `OutOfStock`. The payment service reacts by refunding, and the order service marks the order **CANCELLED** and emails the customer. No step ever held a lock across services.

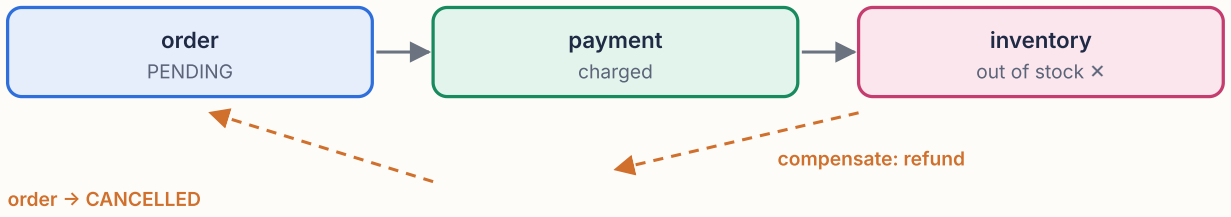


Figure 3. A saga undoes completed steps with compensations when a later step fails.

5 Before you reach for either

Ask whether the data really belongs in different services. If two tables must always change together, they may belong in one service and one database. Distributed transactions are often a sign that service boundaries cut through one consistency boundary.

COMMON MISTAKES

- Calling three services one after another and hoping none fails.
- Writing to the DB and then publishing to Kafka without an outbox.
- Sagas without idempotent steps, so retries charge twice.
- Showing PENDING states to users as if they were final.

6 Key terms

Distributed transaction: one logical change across several data stores

Saga: sequence of local transactions with compensations

Transactional outbox: save data and event in one local transaction

2PC: prepare, then commit, under a coordinator

Compensating action: business undo of a completed step

7 Interview questions

Q1. Why can't microservices just use a normal transaction? BASIC

Each service owns its own database; a database transaction can't span other databases or external APIs, so there's no single commit point.

Q2. 2PC or saga: what's the core trade-off? BASIC

2PC gives atomicity and isolation but blocks and couples participants; a saga avoids locks and coupling but is only eventually consistent and needs compensations.

Q3. Design checkout across order, payment and inventory services. SCENARIO

Use a saga: pending order, charge with an idempotency key, reserve stock; on failure refund and cancel. Publish events through outboxes and deduplicate in consumers.

Q4. When is 2PC still reasonable? SCENARIO

Within one organisation's databases that support XA, short transactions, low latency between them, and when strict atomicity matters more than availability.

Q5. What isolation problems do sagas have? TRICKY

Other transactions can see intermediate states, like a charged but not yet reserved order. Mitigate with semantic locks (PENDING status), commutative updates, and re-checking values.

Q6. Why might needing a distributed transaction indicate a design problem? TRICKY

Data that must change atomically usually belongs to one consistency boundary; splitting it across services creates the problem in the first place.

Next: SD-H-016 · Two-Phase Commit (2PC) backendroadmap.com